Manuel complet du framework Csound–Opusmodus

Stéphane Boussuge

2026-04-10

Contents

1 Avant-propos **6**

2 Philosophie générale **7**

3 Architecture du framework **8**
3.1 Les deux niveaux principaux 8
3.1.1 1. Le noyau de framework (`Csound.lisp`) 8
3.1.2 2. La bibliothèque d'instruments (`CsoundInstrumentsLib.lisp`) 8
3.2 Structures fondamentales 8
3.2.1 `csound-instrument` 8
3.2.2 `csound-event` 9
3.3 Types d'objets 9
3.3.1 `:instrument` 9
3.3.2 `:fx` 10
3.3.3 `:output` 10

4 Chargement et session de travail **10**
4.1 Bibliothèque globale 10
4.2 Gestion du processus Csound temps réel 11

5 La DSL de définition des instruments **11**
5.1 La macro `defcsinstr` 11
5.2 Clauses supportées 12
5.2.1 `(:type ...)` 12
5.2.2 `(:pfields ...)` 12
5.2.3 `(:globals ...)` 12

5.2.4 `(:inputs ...)` 13
5.2.5 `(:outputs ...)` 13
5.2.6 `(:body ...)` 13
5.2.7 `(:doc ...)` 13

6 La mini-DSL Csound dans `:body` **13**
6.1 Lignes brutes 13
6.2 Assignation simple 13
6.3 Op-code général 13
6.4 Sortie vers outlet 14
6.5 Inlet audio 14
6.6 Sortie directe 14
6.7 Opérateurs infixes 14

7 Macros utilitaires de définition **14**
7.1 `def-simple-stereo-instrument` 14
7.2 `def-simple-stereo-fx` 14
7.3 `def-simple-direct-instrument` 15
7.4 `def-simple-output` 15

8 Événements et validation **15**
8.1 `cs-event` 15
8.2 `cs-events` 15
8.3 Validation stricte 15
8.4 Instruments vs effets 16

9 Normalisation des paramètres **16**
9.1 Paramètres textuels 16
9.2 Quotation utilitaire 16

10 Rendu d'événements **16**
10.1 Broadcast et répétition automatique 17

11 Routage audio **17**
11.1 Routes stéréo 17
11.2 Routage par défaut 18
11.3 Activation permanente des effets 18

12 Génération du score avec `def-csound-score` **18**
12.1 Rôle 18
12.2 Paramètres principaux 18
12.2.1 `:write-to-file` 18
12.2.2 `:file` 19
12.2.3 `:instruments` 19
12.2.4 `:fx` 19
12.2.5 `:routes` 19
12.2.6 `:events` 19
12.2.7 `:score-headers` 19
12.2.8 `:global-orchestra-code` 19
12.2.9 `:cs-options` 19
12.2.10 `:csound-bin` 19
12.2.11 `:sr`, `:ksmps`, `:nchnls`, `:dbfs` 19
12.2.12 `:play` 20
12.3 Schéma de rendu 20
12.4 Exemple minimal 20

13 Lecture temps réel **21**

14 Rendu offline en WAV **21**
14.1 Variables utiles 21
14.2 Mémorisation automatique du dernier score 21
14.3 `render-score` 22
14.4 `render-last-score` 22
14.5 Ouverture automatique du WAV 22
14.6 Workflow recommandé 23

15 Helpers de compatibilité : `iblock` **23**
15.1 Rôle 23

16 Gestion des options Csound **23**

17 Conventions d'usage recommandées **24**
17.1 1. Toujours utiliser des chemins absolus 24
17.2 2. Garder `output` comme dernier effet 24
17.3 3. Réserver `cs-event` aux vrais instruments 24
17.4 4. Contrôler la densité énergétique 24
17.5 5. Utiliser le rendu offline pour le diagnostic 25

18 Débogage et problèmes fréquents **25**
18.1 Paramètre inconnu 25
18.2 Instrument inconnu 25
18.3 Silence complet au rendu 25
18.4 `error code 127` 26
18.5 `render-last-score` ne fonctionne pas 26
18.6 Mur vertical dans la waveform 26

19 Catalogue complet des instruments **26**
19.1 Core drones and pads 26
19.1.1 `sinedrone` 26
19.1.2 `sineunitenvelope` 27
19.1.3 `sawdrone` 27
19.1.4 `sawunitenvelope` 27
19.1.5 `night` 27
19.1.6 `pad1` 27
19.1.7 `vco2pad1` 28
19.1.8 `vco2x2mod1` 28
19.1.9 `poscilx2mod1` 28
19.1.10 `cmjanalogpad` 28
19.2 Sampling and file playback 28
19.2.1 `samplerreverb` 28
19.2.2 `samplerraw` 29
19.2.3 `samplerunitenv` 29
19.2.4 `samplerunitenvbp` 29
19.2.5 `samplerunitenvdist` 29
19.2.6 `samplerunitenvpeq` 30
19.2.7 `samplersahenv` 30
19.2.8 `samplercrossenv` 30
19.2.9 `diskin2` 30
19.2.10 `lposcil` 30
19.2.11 `mincer1` 31
19.2.12 `mincer2` 31
19.2.13 `mincer3` 31
19.3 Noise and vocoder/noise resynthesis 31
19.3.1 `noisewhite` 31
19.3.2 `noispitched` 32
19.3.3 `noiseunitenv` 32
19.3.4 `noisetambourine` 32
19.3.5 `noiseunitenvbp` 32

19.3.6 noisesahenv . . . 33
19.3.7 noisesahenvdist . . . 33
19.3.8 vocodenoisesingle . . . 33
19.3.9 vocodenoisesinglegliss . . . 33
19.3.10 vocodenoisequadremap . . . 34
19.3.11 vocodenoisequadscale . . . 34
19.3.12 vocodenoisequadscaleremap . . . 34
19.3.13 vocodenoiseoctscale . . . 34
19.3.14 vocodenoiseoctscaleremap . . . 35
19.3.15 phasevocoderead . . . 35
19.4 FM and resonant/plucked . . . 35
19.4.1 fmbasic . . . 35
19.4.2 fmclarinet . . . 35
19.4.3 fmwooddrum . . . 36
19.4.4 fmstring . . . 36
19.4.5 fm1 . . . 36
19.4.6 synthrezzy . . . 36
19.4.7 synthwaveformvibrato . . . 36
19.4.8 synthvcoaudioenvelopesinequad . . . 37
19.4.9 synthvcoaudioenvelopesquarequad . . . 37
19.4.10 synthvcodistort . . . 37
19.4.11 plucktamhats . . . 37
19.4.12 pluckformant . . . 38
19.4.13 pluckunitenvelope . . . 38
19.4.14 trombone . . . 38
19.5 Analog synthesis . . . 38
19.5.1 ckanalog . . . 38
19.5.2 analog1 . . . 39
19.5.3 analog1b . . . 39
19.5.4 analog1c . . . 39
19.5.5 analog2 . . . 39
19.5.6 analog2b . . . 40
19.5.7 analog2c . . . 40
19.5.8 analog2d . . . 40
19.6 Granular . . . 41
19.6.1 grain1 . . . 41
19.6.2 grain1b . . . 41
19.6.3 grain2 . . . 41
19.6.4 diskgrain1 . . . 41
19.7 FX and output . . . 41

19.7.1 `plateau1` 41

20 Résumé opérationnel **42**

21 Fonctions principales du noyau : aide-mémoire **42**
21.1 Gestion du moteur 42
21.2 Bibliothèque d'instruments 43
21.3 Définition d'instruments 43
21.4 Événements 43
21.5 Rendu 44
21.6 Helpers de rendu offline ajoutés 44
21.7 Helpers divers 44

22 Remarque finale **44**

1 Avant-propos

Ce document décrit de manière exhaustive le framework Csound intégré dans Opusmodus tel qu'il apparaît dans les fichiers `Csound.lisp` et `CsoundInstrumentsLib.lisp`, ainsi que le flux de rendu offline ajouté récemment (`render-score`, `render-last-score`, mémorisation du dernier score écrit).

L'objectif du framework est de fournir un environnement unifié où :

- la composition algorithmique est écrite en Common Lisp / Opusmodus ;
- les instruments et effets Csound sont déclarés par une DSL Lisp ;
- les événements sont générés à partir de paramètres nommés ;
- le routage audio est construit automatiquement ;
- le score Csound final (`.csd`) est généré depuis Lisp ;
- la lecture temps réel et le rendu offline peuvent être pilotés depuis le même environnement.

Ce manuel couvre :

1. l'architecture générale ;
2. la bibliothèque d'instruments ;

3. la DSL de définition des instruments ;
4. la génération d'événements ;
5. le routage et la génération de score ;
6. la lecture temps réel ;
7. le rendu offline en WAV ;
8. les bonnes pratiques et le débogage ;
9. un catalogue complet des instruments et de leurs paramètres.

2 Philosophie générale

Le framework repose sur une idée simple : écrire la structure d'un projet Csound en Lisp, avec le moins possible de texte Csound brut dispersé dans le code de composition.

Au lieu d'écrire directement un `.csd` complet à la main, on procède en plusieurs couches :

1. on enregistre des instruments dans une bibliothèque globale ;
2. on crée des événements avec des paramètres nommés et validés ;
3. on déclare un score avec `def-csound-score` ;
4. le framework rend l'orchestre, le score, les connexions et l'activation des effets ;
5. on joue le résultat ou on le rend offline.

Cette approche offre plusieurs avantages :

- validation structurelle avant rendu ;
- réutilisation des instruments ;
- code compositionnel plus lisible ;
- compatibilité naturelle avec les processus algorithmiques d'Opusmodus ;
- possibilité de générer automatiquement plusieurs variantes d'une pièce.

3 Architecture du framework

3.1 Les deux niveaux principaux

Le framework se divise en deux grands ensembles :

3.1.1 1. Le noyau de framework (`Csound.lisp`)

Il contient principalement :

- les structures de données ;
- la bibliothèque d'instruments ;
- la DSL de rendu Csound ;
- la validation des événements ;
- la génération du `.csd` ;
- la gestion lecture / arrêt / redémarrage de Csound ;
- les helpers de routage ;
- les helpers de rendu offline ajoutés récemment.

3.1.2 2. La bibliothèque d'instruments (`CsoundInstrumentsLib.lisp`)

Elle contient les définitions `defcsinstr` des instruments et effets proprement dits : drones, sampling, granularité, FM, analogique, vocoder, pads, effets, etc.

3.2 Structures fondamentales

Le framework repose principalement sur deux structures :

3.2.1 `csound-instrument`

Chaque instrument enregistré possède les champs suivants :

`name` nom textuel de l'instrument ;

`type` `:instrument`, `:fx` ou `:output` ;

`pfields` liste symbolique des paramètres mappés à `p4...pn` ;

`globals` lignes globales Csound à émettre avant `instr ...` ;

`inputs` noms des inlets audio si l'objet reçoit du signal ;

`outputs` noms des outlets audio si l'objet émet du signal ;

`body` corps de l'instrument sous forme de DSL Lisp ou de chaînes Csound ;

`doc` chaîne de documentation.

3.2.2 `csound-event`

Chaque événement contient :

`instrument` nom de l'instrument ciblé ;

`start` temps de départ ;

`dur` durée ;

`params` plist des paramètres nommés.

3.3 Types d'objets

Le framework distingue clairement trois types :

3.3.1 `:instrument`

Un générateur d'événements, c'est-à-dire un instrument auquel on peut envoyer des `cs-event`.

Exemples :

- `sinedrone`
- `fmstring`
- `vco2pad1`
- `grain1`

3.3.2 `:fx`

Un processeur audio inséré dans une chaîne de routage. Il n'est pas censé recevoir de `cs-event` en score.

Exemples :

- `plateau1`
- `reverberator`
- `compressor`

3.3.3 `:output`

Le dernier maillon de sortie audio. Lui aussi est toujours actif et reçoit le signal final via des inlets.

Exemple :

- `output`

4 Chargement et session de travail

4.1 Bibliothèque globale

Les instruments sont enregistrés dans la table globale :

```
*csound-library*
```

Fonctions utiles :

```
(clear-csound-library)
(list-csound-instruments)
(find-csound-instrument "sinedrone")
(describe-csound-instrument "vco2pad1")
```

- `clear-csound-library` vide la bibliothèque ;
- `list-csound-instruments` liste les noms disponibles ;
- `find-csound-instrument` récupère la structure complète ;
- `describe-csound-instrument` donne un résumé compact.

4.2 Gestion du processus Csound temps réel

Variables globales :

```
*csound-process*
*csound-last-file*
*csound-last-command*
```

Fonctions :

```
(csound-running-p)
(stop-csound)
(restart-csound)
```

Usage :

```
(when (csound-running-p)
  (stop-csound))
```

5 La DSL de définition des instruments

5.1 La macro `defcsinstr`

C'est la macro centrale du système.

Forme générale :

```
(defcsinstr nom
  (:type :instrument)
  (:pfields amp midi pan1 pan2)
  (:globals
   "giSine ftgen 1, 0, 8192, 10, 1")
  (:inputs (leftin rightin))
  (:outputs (leftout rightout))
  (:body
   (kamp = (ampdb p4))
   (outs asig asig))
  (:doc "Documentation de l'instrument."))
```

5.2 Clauses supportées

5.2.1 `(:type ...)`

Valeurs principales :

- `:instrument`
- `:fx`
- `:output`

Si aucune clause `:type` n'est fournie, la macro considère par défaut qu'il s'agit d'un `:instrument`.

5.2.2 `(:pfields ...)`

Liste ordonnée des paramètres utilisateur. Cette liste définit le mapping des paramètres nommés vers `p4, p5, p6...` au moment du rendu des événements.

Exemple :

```
(:pfields amp midi pan1 pan2)
```

Dans ce cas :

- `p4 amp`
- `p5 midi`
- `p6 pan1`
- `p7 pan2`

5.2.3 `(:globals ...)`

Liste de lignes globales à écrire avant l'instrument dans l'orchestre Csound.

Utilisé pour :

- tables d'ondes ;
- globales audio ;
- UDOs éventuelles ;
- variables de configuration.

5.2.4 `(:inputs ...)`

Déclare les noms d'inlets audio. Utile pour les effets et la sortie.

Exemple :

```
(:inputs (leftin rightin))
```

5.2.5 `(:outputs ...)`

Déclare les noms d'outlets audio.

Exemple :

```
(:outputs (leftout rightout))
```

5.2.6 `(:body ...)`

Corps de l'instrument, sous forme de lignes DSL ou de chaînes Csound brutes.

5.2.7 `(:doc ...)`

Documentation libre.

6 La mini-DSL Csound dans `:body`

6.1 Lignes brutes

Une chaîne de caractères est écrite telle quelle :

```
"asig oscili 0.2, 440, 1"
```

6.2 Assignation simple

```
(kamp = (ampdb p4))
```

donne :

```
kamp = ampdb(p4)
```

6.3 Op-code général

```
(asig oscili kamp kcps 1)
```

donne :

```
asig oscili kamp, kcps, 1
```

6.4 Sortie vers outlet

```
(outleta leftout asigl)
(outleta rightout asigr)
```

6.5 Inlet audio

```
(aleftin inleta leftin)
(arightin inleta rightin)
```

6.6 Sortie directe

```
(outs aleftin arightin)
```

6.7 Opérateurs infixes

La DSL sait aussi rendre certaines expressions infixes :

```
(+ a b)
(* asig aenv)
(>= kval 0.5)
```

7 Macros utilitaires de définition

Le framework propose plusieurs macros raccourcies.

7.1 `def-simple-stereo-instrument`

Crée un `:instrument` stéréo avec sorties `leftout/rightout`.

```
(def-simple-stereo-instrument padx
  (amp midi pan1 pan2)
  ...)
```

7.2 `def-simple-stereo-fx`

Crée un `:fx` stéréo avec entrées `leftin/rightin` et sorties `leftout/rightout`.

```
(def-simple-stereo-fx myfx
  ...)
```

7.3 `def-simple-direct-instrument`

Crée un instrument simple sans sorties déclarées automatiquement.

7.4 `def-simple-output`

Crée un `:output` simple.

8 Événements et validation

8.1 `cs-event`

La macro `cs-event` est la manière standard de créer un bloc d'événements.

Exemple :

```
(cs-event "sinedrone"
          :start '(0 5 10)
          :dur   '(8 8 8)
          :amp   '(-24 -26 -28)
          :midi  '(48 55 60)
          :pan1  '(0.2 0.3 0.4)
          :pan2  '(0.8 0.7 0.6))
```

Le framework :

1. normalise les paramètres ;
2. valide qu'ils appartiennent à la liste de `:pfields` de l'instrument ;
3. diffuse les listes à la bonne longueur ;
4. génère autant de lignes `i` que nécessaire.

8.2 `cs-events`

Version fonctionnelle de `cs-event`.

8.3 Validation stricte

Si un paramètre n'est pas déclaré dans l'instrument, le framework signale une erreur.

Exemple de principe :

```
Parameter :freq is not declared in instrument sinedrone.
Allowed parameters: (:AMP :MIDI :PAN1 :PAN2)
```

Cette validation est essentielle pour éviter les erreurs silencieuses.

8.4 Instruments vs effets

Seuls les objets de type `:instrument` peuvent recevoir des `cs-event`. Si on essaie de cibler un `:fx` ou un `:output`, le framework émet une erreur.

9 Normalisation des paramètres

9.1 Paramètres textuels

Le framework reconnaît automatiquement certains pfields comme devant être rendus sous forme de chaînes Csound :

```
*csound-string-pfields*
=> (:file :path :sample :samplefile :filename)
```

Ainsi :

```
:file "/Users/me/sample.wav"
```

sera automatiquement rendu avec guillemets dans le score.

9.2 Quotation utilitaire

Fonctions :

```
(cs-quoted-string "abc")
(cs-already-quoted-string-p ""abc"")
```

10 Rendu d'événements

Le rendu final des événements est assuré par `render-csound-event`.

Principe :

1. récupération de l'instrument ;
2. lecture de l'ordre des pfields ;

3. alignement de toutes les colonnes ;
4. répétition automatique des valeurs scalaires ;
5. émission de lignes Csound :

```
i "sinedrone" 0 8 -24 48 0.2 0.8
i "sinedrone" 5 8 -26 55 0.3 0.7
...
```

10.1 Broadcast et répétition automatique

Les scalaires sont répétés à la longueur maximale des listes. Le calcul se fait via :

- `%repeat-to-length`
- `%max-param-length`

Cela permet des appels très compacts :

```
(cs-event "fmstring"
          :start '(0 3 6 9)
          :dur   6
          :amp   -30
          :midi  '(72 74 76 79)
          :pan1  0.2
          :pan2  0.8)
```

11 Routage audio

11.1 Routes stéréo

Le routage standard se fait par paires :

- `leftout -> leftin`
- `rightout -> rightin`

Fonctions concernées :

```
(make-stereo-route from to)
(make-default-routes instruments fx)
(render-connect-lines routes)
(render-alwayson-lines fx-list)
```

11.2 Routage par défaut

Si aucun routage n'est fourni, `make-default-routes` construit automatiquement :

- chaque instrument → premier effet ;
- puis effet 1 → effet 2 → ... → dernier effet.

Exemple :

```
:instruments '("sinedrone" "fmstring")
:fx '("plateau1" "output")
```

génère implicitement :

- `sinedrone -> plateau1`
- `fmstring -> plateau1`
- `plateau1 -> output`

11.3 Activation permanente des effets

Les effets et la sortie sont démarrés par :

```
alwayson "plateau1"
alwayson "output"
```

Le framework les émet automatiquement via `render-alwayson-lines`.

12 Génération du score avec `def-csound-score`

12.1 Rôle

`def-csound-score` est la fonction centrale de génération d'un fichier `.csd` complet.

12.2 Paramètres principaux

12.2.1 `:write-to-file`

Écrit ou non le fichier sur disque.

12.2.2 `:file`

Chemin du `.csd` à écrire.

Sur macOS, il est fortement recommandé d'utiliser un chemin absolu :

```
:file "/Users/stephaneboussuge/Csoundscores/Mon_Fichier.csd"
```

12.2.3 `:instruments`

Liste des instruments générateurs utilisés dans la pièce.

12.2.4 `:fx`

Liste des effets et de la sortie, dans l'ordre de la chaîne audio.

12.2.5 `:routes`

Routes explicites. Si absent, le framework utilise le routage par défaut.

12.2.6 `:events`

Liste des blocs d'événements.

12.2.7 `:score-headers`

Lignes ajoutées dans `<CsScore>`, par exemple :

```
:score-headers '("f 0 600")
```

12.2.8 `:global-orchestra-code`

Code global ajouté dans `<CsInstruments>` avant les instruments.

12.2.9 `:cs-options`

Chaîne des options Csound, généralement produite par `build-cs-options`.

12.2.10 `:csound-bin`

Chemin vers l'exécutable Csound pour la lecture temps réel.

12.2.11 `:sr`, `:ksmps`, `:nchnls`, `:dbfs`

Paramètres globaux audio.

12.2.12 `:play`

Si non nul, le framework lance immédiatement Csound après écriture du fichier.

12.3 Schéma de rendu

`def-csound-score` produit dans l'ordre :

1. en-tête Csound ;
2. code global projet ;
3. définitions des instruments ;
4. définitions des effets ;
5. lignes de routage ;
6. lignes `alwayson` pour les effets ;
7. ouverture de `<CsScore>` ;
8. score headers ;
9. événements ;
10. fermeture du fichier.

12.4 Exemple minimal

```
(def-csound-score
  :file "/Users/stephaneboussuge/Csoundscores/Test.csd"
  :instruments '("sinedrone")
  :fx '("plateau1" "output")
  :score-headers '("f 0 60")
  :events
  (list
   (cs-event "sinedrone"
             :start '(0 10 20)
             :dur   '(12 12 12)
             :amp   '(-26 -28 -30)
             :midi  '(48 55 60)
             :pan1  '(0.2 0.3 0.4)
             :pan2  '(0.8 0.7 0.6)))
  :play nil)
```

13 Lecture temps réel

Quand `:play t` est donné à `def-csound-score` :

1. si un processus Csound tourne déjà, il est arrêté ;
2. le fichier courant est mémorisé dans `*csound-last-file*` ;
3. la commande est mémorisée dans `*csound-last-command*` ;
4. un nouveau processus est lancé avec `uiop:launch-program`.

C'est la manière la plus directe d'auditionner rapidement un score.

14 Rendu offline en WAV

Cette section décrit l'extension récente ajoutée à ton framework.

14.1 Variables utiles

```
*csound-render-output-directory*
*last-csound-score-file*
```

- `*csound-render-output-directory*` : dossier de rendu WAV ;
- `*last-csound-score-file*` : dernier `.csd` écrit, utilisé par `render-last-score`.

14.2 Mémorisation automatique du dernier score

Après écriture du fichier dans `def-csound-score`, il est recommandé de mémoriser :

```
(setf *last-csound-score-file*
      (namestring (truename file)))
```

Cela garantit un chemin absolu valide.

14.3 `render-score`

Cette fonction rend un `.csd` vers un WAV.

Exemple :

```
(render-score
 "/Users/stephaneboussuge/Csoundscores/Lumina_JI_Study.csd"
 :print-command t)
```

Fonctionnement :

1. résout le chemin du `.csd` ;
2. construit automatiquement le chemin de sortie WAV si besoin ;
3. lance Csound en offline ;
4. peut éventuellement ouvrir automatiquement le fichier rendu.

14.4 `render-last-score`

Cette fonction rend le dernier score mémorisé.

Exemple :

```
(render-last-score)
```

ou :

```
(render-last-score :print-command t :open t)
```

14.5 Ouverture automatique du WAV

Si tu as intégré l'option `:open` dans `render-score` et `render-last-score`, le fichier WAV peut s'ouvrir automatiquement après rendu.

Sur macOS, cela passe généralement par :

```
(uiop:run-program (list "open" out) ...)
```

ou, pour QuickTime explicitement :

```
(uiop:run-program
 (list "open" "-a" "QuickTime Player" out)
 ...)
```

14.6 Workflow recommandé

```
(def-csound-score
  :file "/Users/stephaneboussuge/Csoundscores/Lumina_JI_Study.csd"
  :instruments '("sinedrone" "night")
  :fx '("plateau1" "output")
  :score-headers '("f 0 600")
  :events ...
  :play nil)

(render-last-score :open t)
```

15 Helpers de compatibilité : `iblock`

15.1 Rôle

`iblock` est un utilitaire de diffusion de colonnes.

Exemple :

```
(iblock '("fmtest")
        '(0 1 2)
        '(0.5)
        '(-12)
        '(60 64 67))
```

Le résultat peut être converti en lignes Csound par :

```
(iblock-to-csound-lines ...)
```

Cette partie est utile surtout pour certains workflows plus anciens ou pour de la construction de matériau de score très compacte.

16 Gestion des options Csound

Le framework prévoit une construction des options via `build-cs-options` et une config utilisateur dans `*csound-config*`.

Dans la pratique, cela sert à centraliser :

- le chemin de l'exécutable Csound ;
- les variables d'environnement utiles ;

- les répertoires de samples ;
- les options de rendu et de lecture.

17 Conventions d'usage recommandées

17.1 1. Toujours utiliser des chemins absolus

Préférer :

```
"/Users/stephaneboussuge/Csoundscores/Mon_Fichier.csd"
```

17.2 2. Garder `output` comme dernier effet

Chaîne type :

```
:fx '("plateau1" "output")
```

ou :

```
:fx '("compressor" "plateau1" "output")
```

17.3 3. Réserver `cs-event` aux vrais instruments

Ne jamais envoyer d'événements à un `:fx` ou à `output`.

17.4 4. Contrôler la densité énergétique

Avec les textures sinusoïdales longues, la somme des voix peut devenir énergétiquement instable. Le framework évite de nombreuses erreurs structurelles, mais pas toutes les situations de surcharge musicale.

Bonne pratique :

- limiter le nombre de voix simultanées ;
- désynchroniser légèrement les départs ;
- éviter les empilements exacts de fréquences identiques ;
- surveiller les limiteurs de sortie.

17.5 5. Utiliser le rendu offline pour le diagnostic

Le rendu offline est le meilleur moyen de :

- vérifier la forme d'onde ;
- détecter les surcharges ;
- comparer plusieurs variantes.

18 Débogage et problèmes fréquents

18.1 Paramètre inconnu

Symptôme :

```
Parameter :freq is not declared in instrument sinedrone
```

Cause : l'instrument déclare un pfield différent, par exemple `midi` au lieu de `freq`.

18.2 Instrument inconnu

Symptôme :

```
Unknown Csound instrument: pad1
```

Cause possible : bibliothèque non chargée ou instrument non enregistré.

18.3 Silence complet au rendu

Causes possibles :

- effets/sortie non activés ;
- routage absent ;
- score vide ;
- mauvais chemin de fichier de sample ;
- rendu du mauvais `.csd`.

18.4 `error code 127`

Cause typique : exécutable Csound introuvable. La solution est d'utiliser le chemin absolu du binaire, via :

```
(getf *csound-config* :csound-bin)
```

18.5 `render-last-score` ne fonctionne pas

Cause typique : `*last-csound-score-file*` contient un chemin relatif ou ancien.

Solution : mémoriser le fichier via :

```
(setf *last-csound-score-file*
      (namestring (truename file)))
```

18.6 Mur vertical dans la waveform

Cause probable : accumulation énergétique excessive dans une zone du score, souvent avec des sinusoïdes longues, des fréquences corrélées, et un limiteur final qui écrase massivement.

19 Catalogue complet des instruments

Ce catalogue est organisé par familles. Pour chaque instrument sont indiqués :

- les pfields disponibles ;
- la documentation embarquée quand elle existe.

19.1 Core drones and pads

19.1.1 `sinedrone`

Type `instrument`

Pfields `amp midi pan1 pan2`

Description Sine drone borrowed from Christopher Ariza AthenaCL.

19.1.2 `sineunitenvelope`

Type `instrument`

Pfields `amp midi pan1 pan2 suspcent suscenterpcent`

Description Sine with unit-envelope style shaping.

19.1.3 `sawdrone`

Type `instrument`

Pfields `amp midi pan1 pan2`

Description Saw drone borrowed from Christopher Ariza AthenaCL.

19.1.4 `sawunitenvelope`

Type `instrument`

Pfields `amp midi pan1 pan2 suspcent suscenterpcent`

Description Saw with unit-envelope style shaping.

19.1.5 `night`

Type `instrument`

Pfields `amp midi pan1 pan2`

Description String-pad borrowed from Bay at Night.

19.1.6 `pad1`

Type `instrument`

Pfields `amp midi rise dec wave`

Description Versatile pad using an externally defined wavetable.

19.1.7 `vco2pad1`

Type `instrument`

Pfields `amp freq atk rel detune bright vibdepth vibrate submix pan1 pan2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.1.8 `vco2x2mod1`

Type `instrument`

Pfields `amp freq atk rel detune bright vibdepth vibrate pwm submix noisemix drift pan1 pan2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.1.9 `poscilx2mod1`

Type `instrument`

Pfields `amp freq atk rel detune bright vibdepth vibrate submix noisemix drift osc1fn osc2fn subfn vibfn pan1 pan2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.1.10 `cmjanalogpad`

Type `instrument`

Pfields `amp midi pan1 pan2`

Description AnalogPad adapted faithfully from Comajuncosas AnalogPad CSD version, using MIDI notes and k-rate stereo panning.

19.2 Sampling and file playback

19.2.1 `samplerreverb`

Type `instrument`

Pfields `file amp midi pan1 pan2 skiptime atk rel rvbtime rvbgain`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.2.2 `samplerraw`

Type `instrument`

Pfields `file amp midi pan1 pan2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.2.3 `samplerunitenv`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center cutoff1 cutoff2 q skiptime`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.2.4 `samplerunitenvbp`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center cf1 cf2 bw1 bw2 skiptime`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.2.5 `samplerunitenvdist`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center distin distout curvepos curveneg lpf1 lpf2 lpfq skiptime`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.2.6 `samplerunitenvpeq`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center cf1 cf2 q1 q2 gain1 gain2 filtertype skiptime`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.2.7 `samplersahenv`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center lpf1 lpf2 lpfq skiptime hpf1 hpf2 rate1lo rate1hi rate2lo rate2hi rate3`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.2.8 `samplercrossenv`

Type `instrument`

Pfields `filea fileb amp midi pan1 pan2 sustain center lpf1 lpf2 lpfq skipa skipb fftsize bias1 bias2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.2.9 `diskin2`

Type `instrument`

Pfields `file amp speed skiptime wrap atk rel pan1 pan2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.2.10 `lposcil`

Type `instrument`

Pfields `file amp speed loopstart loopend atk rel pan1 pan2`

Description Looping audio file player based on lposcil. The file is passed in :file, loop points are in samples, and mono/stereo files are handled automatically.

19.2.11 `mincer1`

Type `instrument`

Pfields `amp file pos1 pos2 pitch lock fftsize decim pan1 pan2`

Description Mincer1: mono file playback via ftgenonce + mincer, with k-rate panning.

19.2.12 `mincer2`

Type `instrument`

Pfields `amp file startpos speed pitch lock fftsize decim freeze jitterdepth jitterrate att hold dec sus rel cutoff res filttype panmode pan1 pan2 panrate pandepth`

Description Mincer2: file-based mincer instrument with AHDSR amplitude envelope, speed scan, freeze, position jitter, post-filter, and manual/autopan/random k-rate panning.

19.2.13 `mincer3`

Type `instrument`

Pfields `amp file startpos speed pitch lock fftsize decim freeze jitterdepth jitterrate looplen xfade att hold dec sus rel cutoff res filttype panmode pan1 pan2 panrate pandepth`

Description Mincer3: file-based mincer instrument with local loop window, crossfaded loop read, AHDSR amplitude envelope, speed, freeze, jitter, post-filter, and manual/autopan/random k-rate panning.

19.3 Noise and vocoder/noise resynthesis

19.3.1 `noisewhite`

Type `instrument`

Pfields `amp midi pan1 pan2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.2 `noispitched`

Type `instrument`

Pfields `amp midi pan1 pan2 cf bw`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.3 `noiseunitenv`

Type `instrument`

Pfields `amp midi pan1 pan2 sustain center cutoff1 cutoff2 q`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.4 `noisetambourine`

Type `instrument`

Pfields `amp midi pan1 pan2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.5 `noiseunitenvbp`

Type `instrument`

Pfields `amp midi pan1 pan2 sustain center cf1 cf2 bw1 bw2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.6 `noisesahenv`

Type `instrument`

Pfields `amp midi pan1 pan2 sustain center lpf1 lpf2 lpfq hpf1 hpf2 rate1lo rate1hi rate2lo rate2hi rate3`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.7 `noisesahenvdist`

Type `instrument`

Pfields `amp midi pan1 pan2 sustain center lpf1 lpf2 lpfq hpf1 hpf2 rate1lo rate1hi rate2lo rate2hi rate3 rescf1 rescf2 res1 res2 dist1 dist2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.8 `vocodenoisesingle`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center skiptime analysiscutoff bw analysiscf gencf`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.9 `vocodenoisesinglegliss`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center skiptime analysiscutoff bw analysiscf gencf1 gencf2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.10 `vocodenoisequadremap`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center skiptime speed analysiscutoff bw acf1 acf2 acf3 acf4 gcf1 gcf2 gcf3 gcf4 lpf1 lpf2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.11 `vocodenoisequadscale`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center skiptime speed analysiscutoff bw analysisbase analysisscalar genbase genscalar lpf1 lpf2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.12 `vocodenoisequadscaleremap`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center skiptime speed analysiscutoff bw analysisbase analysisscalar genbase genscalar src1 src2 src3 src4 post1 post2 post3 post4 lpf1 lpf2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.13 `vocodenoiseoctscale`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center skiptime speed analysiscutoff bw analysisbase analysisscalar genbase genscalar lpf1 lpf2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.14 `vocodenoiseoctscaleremap`

Type `instrument`

Pfields `file amp midi pan1 pan2 sustain center skiptime speed analysiscutoff bw analysisbase analysisscalar genbase genscalar src1 src2 src3 src4 src5 src6 src7 src8 post1 post2 post3 post4 post5 post6 post7 post8 lpf1 lpf2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.3.15 `phasevocoderead`

Type `instrument`

Pfields `analysis amp midi pan1 pan2 bin start fadein fadeout`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.4 FM and resonant/plucked

19.4.1 `fmbasic`

Type `instrument`

Pfields `amp midi pan1 pan2 factor index`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.4.2 `fmclarinet`

Type `instrument`

Pfields `amp midi pan1 pan2 imax`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.4.3 `fmwooddrum`

Type `instrument`

Pfields `amp midi pan1 pan2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.4.4 `fmstring`

Type `instrument`

Pfields `amp midi pan1 pan2 rise dec vibdel vibwth vibrate`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.4.5 `fm1`

Type `instrument`

Pfields `amp midi mod index1 index2 rise dec pan1 pan2`

Description Simple FM instrument.

19.4.6 `synthrezzy`

Type `instrument`

Pfields `amp freq sweep rez wave drive pan1 pan2`

Description Resonant synth with selectable waveform, filter sweep and controllable drive.

19.4.7 `synthwaveformvibrato`

Type `instrument`

Pfields `amp freq atk rel vibdepth vibdelay vibrate wave1 wave2 xfade pan1 pan2`

Description Waveform-morphing synth with delayed vibrato and layered detuned oscillators.

19.4.8 `synthvcoaudioenvelopesinequad`

Type `instrument`

Pfields `amp freq suspct suscenter lpfstart lpfend lpfq wave widthstart widthend trem1start trem1end trem1amp trem2start trem2end trem2amp trem3start trem3end trem3amp trem4start trem4end trem4amp pan1 pan2`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

19.4.9 `synthvcoaudioenvelopesquarequad`

Type `instrument`

Pfields `amp freq suspct suscenter lpfstart lpfend lpfq wave widthstart widthend trem1start trem1end trem1amp trem2start trem2end trem2amp trem3start trem3end trem3amp trem4start trem4end trem4amp pan1 pan2`

Description VCO instrument with proportional macro envelope, low-pass filter and four square-based audio-rate tremolo modulators.

19.4.10 `synthvcodistort`

Type `instrument`

Pfields `amp freq suspct suscenter lpfstart lpfend lpfq wave widthstart widthend rcfstart rcfend resstart resend diststart distend pan1 pan2`

Description VCO instrument with proportional macro envelope, resonant lpf18 distortion stage and final low-pass filtering.

19.4.11 `plucktamhats`

Type `instrument`

Pfields `amp freq pan parm lpfreq`

Description Percussive sound somewhere between tam-tam and high-hat.

19.4.12 `pluckformant`

Type `instrument`

Pfields `amp freq pan pluckamp pluckdur fmamp fmrise fmdec index vibdepth vibrate formantamp formantrise`

Description A pluck that slowly morphs into a vocal, formant derived sound.

19.4.13 `pluckunitenvelope`

Type `instrument`

Pfields `amp freq pan function method parm1 parm2 suspct suscenter lpfstart lpfend lpfq`

Description A single pluck with a unit envelope and variable low pass filter.

19.4.14 `trombone`

Type `instrument`

Pfields `amp note1 note2`

Description Trombone derived from Iain McCurdy's Csound Haiku I.

19.5 Analog synthesis

19.5.1 `ckanalog`

Type `instrument`

Pfields `amp midi porta vco2ratio detunehz vcf1 vcf2 rez wav1 wav2 lfowave lforate1 lforate2 ring pan1 pan2`

Description Cook-style analog synth: 2 VCO, 2 LFO PWM, glide, ring-mod, resonant filter, stereo k-rate pan. Best used monophonically for authentic glide behaviour.

19.5.2 `analog1`

Type `instrument`

Pfields `amp midi wave pw att hold dec sus rel cutoff res filttype pan1 pan2`

Description Analog1: single-vco2 synth with AHDSR and selectable LP/BP/HP filter.

19.5.3 `analog1b`

Type `instrument`

Pfields `amp midi wave pw porta att hold dec sus rel fatt fhold fdec fsus frel cutoff envamt res filttype lforate lfodepth drive pan1 pan2`

Description Analog1b: one-vco2 synth with amp/filter envelopes, drive, selectable LP/BP/HP filter, cutoff LFO and portamento.

19.5.4 `analog1c`

Type `instrument`

Pfields `amp midi wave pw pwmrate pwmdepth porta submix noisemix att hold dec sus rel fatt fhold fdec fsus frel cutoff envamt res filttype penvamt patt phold pdec lforate lfodepth drive pan1 pan2`

Description Analog1c: vco2 synth with sub oscillator, noise, PWM, amp/filter AHDSR, pitch envelope, selectable LP/BP/HP filter, drive, filter LFO and portamento.

19.5.5 `analog2`

Type `instrument`

Pfields `amp midi wave1 pw1 wave2 pw2 detune att hold dec sus rel cutoff res filttype pan1 pan2`

Description Analog2: two-vco2 synth with selectable LP/BP/HP filter, AHDSR envelope, and VCO2 detune in cents.

19.5.6 `analog2b`

Type `instrument`

Pfields `amp midi wave1 pw1 pwmrate1 pwmdepth1 wave2 pw2 pwmrate2 pwmdepth2 vco2interval detune mix2 porta att hold dec sus rel fatt fhold fdec fsus frel cutoff envamt res filttype lforate lfodepth drive pan1 pan2`

Description Analog2b: dual-vco2 synth with independent PWM, VCO2 interval and detune, amp/filter AHDSR, selectable LP/BP/HP filter, drive, cutoff LFO and portamento.

19.5.7 `analog2c`

Type `instrument`

Pfields `amp midi wave1 pw1 pwmrate1 pwmdepth1 wave2 pw2 pwmrate2 pwmdepth2 vco2interval detune mix2 porta submix subwave noisemix att hold dec sus rel fatt fhold fdec fsus frel cutoff envamt res filttype penvamt patt phold pdec lforate lfodepth drive pan1 pan2`

Description Analog2c: dual-vco2 synth with interval/detune, independent PWM, sub osc, noise, amp/filter AHDSR, pitch envelope, selectable LP/BP/HP filter, drive, cutoff LFO and portamento.

19.5.8 `analog2d`

Type `instrument`

Pfields `amp midi wave1 pw1 pwmrate1 pwmdepth1 wave2 pw2 pwmrate2 pwmdepth2 vco2interval detune mix2 porta submix subwave noisemix att hold dec sus rel fatt fhold fdec fsus frel cutoff envamt res filttype penvamt patt phold pdec lforate lfodepth drive fmamt syncamt pan1 pan2`

Description Analog2d: dual-vco2 synth with interval/detune, PWM, sub, noise, pitch env, amp/filter AHDSR, selectable LP/BP/HP filter, drive, cutoff LFO, light k-rate FM, sync-like shaping, and portamento.

19.6 Granular

19.6.1 `grain1`

Type `instrument`

Pfields `amp ft dens1 dens2 rise dec pan1 pan2`

Description Granulator for soundfiles loaded in ftables.

19.6.2 `grain1b`

Type `instrument`

Pfields `amp ft dens1 dens2 rise dec pan1 pan2 pitchfact`

Description Granulator for soundfiles with fixed pitch-factor control.

19.6.3 `grain2`

Type `instrument`

Pfields `amp midi fn gdur ovrlp rise dec rndvarfrq1 rndvarfrq2 pan1 pan2`

Description Granulator for generated ftables, not soundfiles.

19.6.4 `diskgrain1`

Type `instrument`

Pfields `file amp freq pitch grsize prate envfn overlaps maxgrsize offset pan1 pan2`

Description Synchronous granular synthesis from a soundfile using the Csound diskgrain opcode.

19.7 FX and output

19.7.1 `plateau1`

Type `fx`

Pfields `amp mix predelay size decay damping diffusion modrate moddepth width`

Description Pas de documentation embarquée dans la définition ; se référer au comportement et aux pfields.

20 Résumé opérationnel

Le workflow minimal complet est le suivant :

```
(def-csound-score
  :file "/Users/stephaneboussuge/Csoundscores/Mon_Fichier.csd"
  :instruments '("sinedrone" "night")
  :fx '("plateau1" "output")
  :score-headers '("f 0 120")
  :events
  (list
   (cs-event "sinedrone"
             :start '(0 8 16)
             :dur   '(12 12 12)
             :amp   '(-26 -28 -30)
             :midi  '(48 55 60)
             :pan1  '(0.2 0.3 0.4)
             :pan2  '(0.8 0.7 0.6)))
  :play nil)

(render-last-score :open t)
```

Ce framework est particulièrement puissant parce qu'il unit :

- la formalisation structurelle de Lisp ;
- la souplesse compositionnelle d'Opusmodus ;
- la précision de synthèse et de traitement de Csound ;
- un workflow de rendu direct utilisable comme véritable environnement de recherche compositionnelle.

21 Fonctions principales du noyau : aide-mémoire

21.1 Gestion du moteur

- `csound-running-p`

- `stop-csound`
- `restart-csound`

21.2 Bibliothèque d'instruments

- `clear-csound-library`
- `register-csound-instrument`
- `find-csound-instrument`
- `list-csound-instruments`
- `describe-csound-instrument`

21.3 Définition d'instruments

- `defcsinstr`
- `def-simple-stereo-instrument`
- `def-simple-stereo-fx`
- `def-simple-direct-instrument`
- `def-simple-output`
- `make-defcsinstr-form`

21.4 Événements

- `cs-event`
- `cs-events`
- `validate-csound-event`
- `normalize-csound-params`

21.5 Rendu

- `render-csound-instrument`
- `render-csound-header`
- `render-csound-event`
- `make-stereo-route`
- `make-default-routes`
- `render-connect-lines`
- `render-alwayson-lines`
- `def-csound-score`

21.6 Helpers de rendu offline ajoutés

- `render-score`
- `render-last-score`
- `csd->default-wav-path`
- `ensure-parent-directory-exists`
- `*last-csound-score-file*`

21.7 Helpers divers

- `iblock`
- `iblock-to-csound-lines`
- `cs-quoted-string`
- `maybe-cs-string`

22 Remarque finale

Ce manuel décrit l'état du framework tel qu'il ressort du code disponible et des extensions récentes ajoutées pendant la session de travail. Si la bibliothèque évolue, le plus efficace sera de régénérer automatiquement l'annexe d'instruments à partir des définitions `defcsinstr`, afin que la documentation reste strictement synchronisée avec le code.